

Technical Architecture & 6-Person Execution Contract

Below is the architecture I recommend for **our actual AI campus-planning project**. I am deliberately keeping it practical for a 6-person college team: one deployable application, one PostgreSQL database, Python ML/optimization modules, React frontend, and Blender Python for 3D generation.

Core principle:

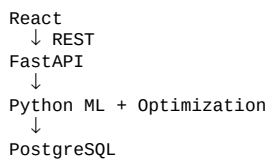
AI decides WHERE things go → Optimization decides WHICH plans are feasible/best → Frontend lets the manager choose → Blender decides HOW the selected plan is visualized in 3D.

0. Important Assumptions

These are the few areas where the existing project description leaves implementation choices open.

Assumption 1 — V1 is a single application

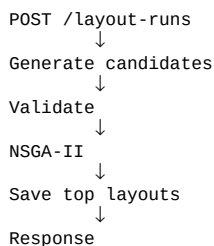
We will **not** build microservices.



ML and optimization are Python modules imported by the backend.

Assumption 2 — Layout generation is initially synchronous

For the college prototype:



If generation becomes slow later, we can introduce Celery/Redis/background jobs.

Assumption 3 — Blender is not a web server

Blender receives a blueprint JSON/file and generates the 3D environment.

We do **not** need a Blender REST API for V1.

Assumption 4 — Synthetic training data

There is no sufficiently large real-world campus-layout dataset available to us, so Person 1 creates synthetic campus configurations and optimizer-generated training targets.

Assumption 5 — Coordinates

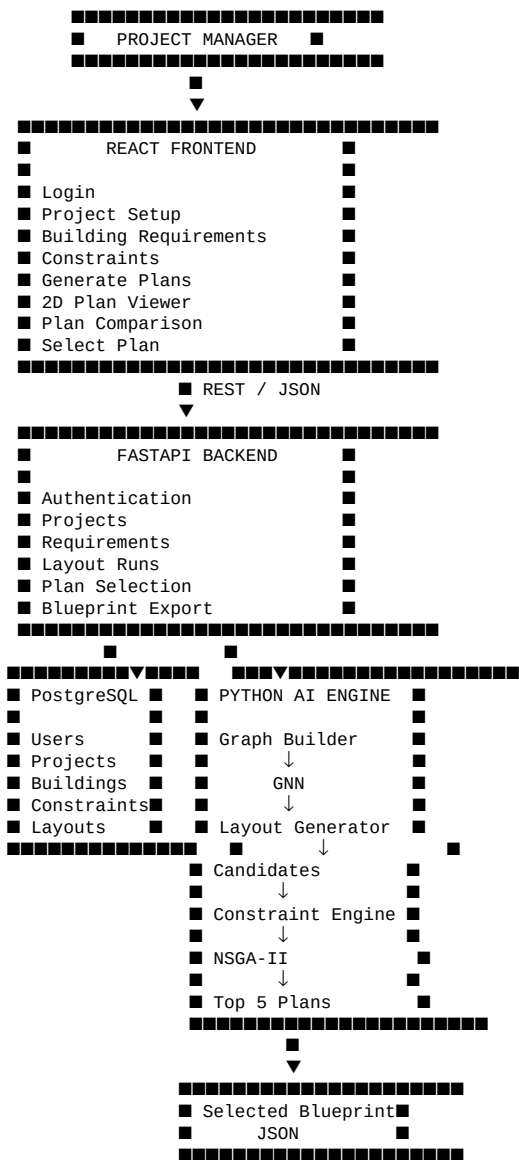
All 2D coordinates use:

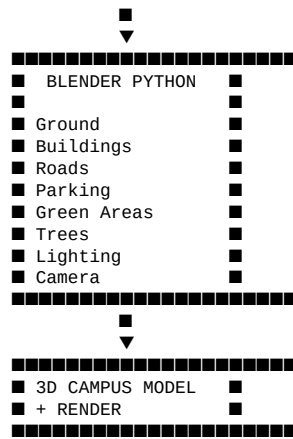
```
Origin: bottom-left of campus plot
X: east/west
Y: north/south
Units: metres
Rotation: degrees
```

Blender converts these directly into its coordinate system.

1. COMPLETE SYSTEM ARCHITECTURE

1.1 Architecture diagram

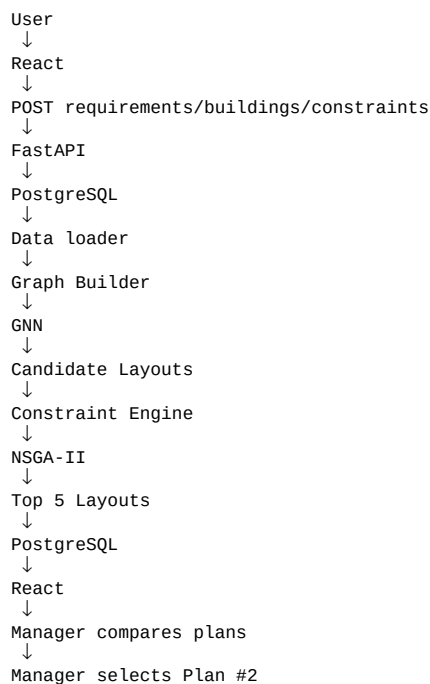




1.2 Major modules

ModuleOwner	
-----	-----
Data/schema + synthetic dataset	Person 1
GNN + layout generation	Person 2
Constraints + NSGA-II	Person 3
FastAPI + PostgreSQL + orchestration	Person 4
React + 2D planning UI	Person 5
Blender procedural 3D	Person 6

1.3 Complete data flow



↓
Blueprint JSON
↓
Blender Python
↓
3D Campus

1.4 Technology stack

Frontend

React
TypeScript
Vite
Axios
SVG/Canvas

Backend

Python
FastAPI
Pydantic
SQLAlchemy
JWT
Uvicorn

AI

PyTorch
PyTorch Geometric
NumPy
Pandas

Optimization

Python
pymoo / NSGA-II
Shapely where useful for geometry

Database

PostgreSQL

3D

Blender
Blender Python API

Infrastructure

Git/GitHub
Docker Compose
PostgreSQL container
Python virtual environment
Node.js

Blender installed locally

1.5 External services/APIs

For V1:

None are required.

That is intentional.

No:

- Google Maps API
- OpenStreetMap API
- Firebase
- AWS
- external AI API
- paid architectural API

The project should work offline after dependencies are installed.

1.6 Authentication flow

```
React
↓
POST /api/auth/login
↓
FastAPI validates credentials
↓
JWT generated
↓
React receives token
↓
Authorization: Bearer <token>
↓
Protected API
```

Only the backend handles password hashing and JWT verification.

2. SIX-PERSON WORK SPLIT

PERSON 1 — DATASET & CAMPUS REPRESENTATION ENGINEER

A. Exact Responsibility

Own the representation of a campus-planning problem.

Person 1 builds the layer that answers:

"How do we convert a real campus requirement into structured data that an ML model can understand?"

They own:

- campus schema
- building schema
- zone schema
- relationship representation
- synthetic campus generation
- training dataset
- graph construction
- preprocessing

They do **not** own the GNN.

B. Features

Feature	What they implement	Priority
-----	-----	--
Campus schema	Site + requirements structure	P0
Building schema	Building attributes	P0
Zone schema	Academic/residential/etc.	P0
Spatial relations	Near/far/access relations	P0
Synthetic generator	Generate campus examples	P0
Dataset validator	Detect malformed data	P0
Graph builder	Convert campus to graph	P0
Dataset export	JSON/PT files	P0
Dataset statistics	Size/distribution	P1

C. Files

```
ml/
├── data/
│   ├── schemas.py           P1
│   ├── campus_schema.py     P1
│   ├── building_schema.py   P1
│   ├── constraint_schema.py P1
│   └── relationships.py     P1
├── dataset/
│   ├── generator.py         P1
│   ├── preprocessing.py     P1
│   ├── validation.py        P1
│   └── graph_builder.py     P1
└── datasets/
    └── README.md            P1
```

D. Inputs

Input: campus requirements

Source: Person 4

Format: JSON/object

```
{
  "projectId": 1,
  "siteWidth": 300,
  "siteHeight": 300,
  "minGreenPercent": 25,
  "minParkingPercent": 10,
  "minRoadWidth": 8,
  "minBuildingGap": 10
}
```

Input: buildings

```
{
  "id": 1,
  "name": "Academic Block A",
  "type": "academic",
  "zone": "academic",
  "width": 60,
  "depth": 40,
  "height": 18,
  "floorCount": 4,
  "requiredCount": 1
}
```

Input: constraints

```
{
  "id": 1,
  "type": "MIN_DISTANCE",
  "sourceId": 1,
  "targetId": 5,
  "value": 100,
  "priority": "hard"
}
```

E. Outputs

Graph

Destination: Person 2

```
CampusGraph(
  node_features,
  edge_index,
  edge_features
)
```

Dataset record

```
{
  "input": {
    "siteWidth": 300,
    "siteHeight": 300
  },
  "nodes": [],
  "edges": [],
  "targetLayout": []
}
```

F. APIs they consume

GET /api/projects/{projectId}/requirements

Headers:

Authorization: Bearer <token>

Response:

```
{
  "projectId": 1,
  "siteWidth": 300,
  "siteHeight": 300,
  "minGreenPercent": 25,
  "minParkingPercent": 10,
  "minRoadWidth": 8,
  "minBuildingGap": 10,
  "entrances": []
}
```

GET /api/projects/{projectId}/buildings

Response:

```
{
  "buildings": [
    {
      "id": 1,
      "name": "Academic Block A",
      "type": "academic",
      "zone": "academic",
      "width": 60,
      "depth": 40,
      "height": 18,
      "floorCount": 4
    }
  ]
}
```

GET /api/projects/{projectId}/constraints

Response:

```
{
  "constraints": []
}
```

Errors:

401 UNAUTHORIZED
403 FORBIDDEN
404 PROJECT_NOT_FOUND

G. APIs they provide

None.

Their principal interface is:

build_graph(campus_data) -> CampusGraph

H. Database

Owns

No database writes.

Reads

projects
campus_requirements
buildings
constraints

Person 4 remains database owner.

I. Dependencies

Depends on	Requires
-----	-----
Person 4	Requirements/buildings/constraints
Person 3	Nothing directly
Person 2	Must provide graph

J. Mock interface

Person 1 can use:

```
{
  "siteWidth": 300,
  "siteHeight": 300,
  "buildings": [
    {
```

```

    "id": 1,
    "type": "academic",
    "zone": "academic",
    "width": 60,
    "depth": 40
  },
  {
    "id": 2,
    "type": "library",
    "zone": "academic",
    "width": 30,
    "depth": 25
  },
  {
    "id": 3,
    "type": "hostel",
    "zone": "residential",
    "width": 50,
    "depth": 30
  }
]
}

```

K. Deliverables

- all schemas
- synthetic generator
- minimum 1,000 test/training samples
- preprocessing
- graph builder
- dataset validator
- documentation
- sample dataset

L. Definition of Done

Person 2 can run:

```
graph = build_graph(campus)
```

and receive a valid graph without manually modifying the data.

M. Tests

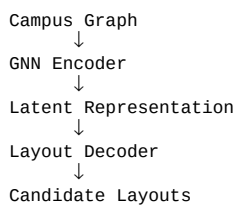
- schema validation
- invalid dimensions
- duplicate building IDs
- graph node count
- graph edge validity
- dataset serialization
- dataset loading

- malformed input handling

PERSON 2 — GNN & LAYOUT GENERATION ENGINEER

A. Exact Responsibility

Own the deep-learning model.



B. Features

FeaturePriority	
-----	--
GNN encoder	P0
Node embeddings	P0
Layout decoder	P0
x/y prediction	P0
rotation prediction	P0
training	P0
checkpoint	P0
inference	P0
candidate generation	P0
evaluation	P1

C. Files

```

ml/
  ■■■ models/
  ■   ■■■ gnn_encoder.py
  ■   ■■■ layout_decoder.py
  ■   ■■■ campus_model.py
  ■
  ■■■ training/
  ■   ■■■ train.py
  ■   ■■■ loss.py
  ■   ■■■ evaluate.py
  ■
  ■■■ inference/
  ■   ■■■ generate.py
  
```

All are Person 2 owned.

D. Inputs

From Person 1:

CampusGraph

Example:

```
{
  "node_features": [[...], [...]],
  "edge_index": [[0,1], [1,2]],
  "edge_features": [[...]]
}
```

E. Outputs

```
{
  "candidateId": "candidate-001",
  "siteWidth": 300,
  "siteHeight": 300,
  "buildings": [
    {
      "buildingId": 1,
      "x": 72.4,
      "y": 120.8,
      "rotation": 0
    }
  ]
}
```

F. APIs consumed

None over HTTP.

Python contract:

```
build_graph(...)
generate_candidates(...)
```

G. APIs provided

```
generate_candidates(
  campus_graph: CampusGraph,
  num_candidates: int
) -> list[CandidateLayout]
```

Expected:

```
generate_candidates(graph, 100)
```

returns exactly 100 candidates unless generation fails.

H. Database

None.

I. Dependencies

Person 1

Requires:

```
CampusGraph
```

Person 3

Provides:

```
CandidateLayout[]
```

J. Mock

```
mock_graph = CampusGraph(  
    node_features=[[0.1, 0.3, 0.5]],  
    edge_index=[[0], [0]],  
    edge_features=[[1.0]]  
)
```

Mock output:

```
[  
  {  
    "candidateId": "mock-001",  
    "siteWidth": 300,  
    "siteHeight": 300,  
    "buildings": [  
      {  
        "buildingId": 1,  
        "x": 50,  
        "y": 80,  
        "rotation": 0  
      }  
    ]  
  }  
)
```

K. Deliverables

- GNN
- decoder
- loss
- training
- checkpoint
- inference
- candidate generation
- evaluation
- documentation

L. Definition of Done

A saved model can be loaded and used to generate candidate layouts from unseen campus graphs.

M. Tests

- tensor dimensions
- forward pass
- training smoke test
- checkpoint load
- inference
- output schema
- reproducibility with fixed seed

PERSON 3 — CONSTRAINT & MULTI-OBJECTIVE OPTIMIZATION ENGINEER

A. Exact Responsibility

Own everything that decides whether a generated layout is actually useful.

```
Candidate layouts
↓
Constraint validation
↓
Objective calculation
↓
NSGA-II
↓
Pareto-ranked layouts
```

B. Features

FeaturePriority	
-----	--
Boundary check	P0
Overlap detection	P0
Minimum gap	P0
Zoning	P0
Road width	P0
Entrance access	P0
Green-space requirement	P0
Parking requirement	P0
Accessibility score	P1
Land utilization	P0
Road efficiency	P1
NSGA-II	P0
Pareto ranking	P0

C. Files

```
optimization/
  constraints/
    boundary.py
    overlap.py
    distance.py
    zoning.py
    roads.py
    green.py
    parking.py
  validator.py
  objectives.py
  nsga2.py
  ranking.py
  optimizer.py
```

D. Inputs

Candidate layouts

From Person 2.

```
{
  "candidateId": "candidate-001",
  "siteWidth": 300,
  "siteHeight": 300,
  "buildings": []
}
```

Requirements

From Person 1/data layer.

```
{
  "minGreenPercent": 25,
  "minParkingPercent": 10,
  "minRoadWidth": 8,
  "minBuildingGap": 10
}
```

E. Outputs

```
{
  "candidateId": "candidate-001",
  "rank": 1,
  "feasible": true,
  "metrics": {
    "landUtilization": 0.72,
    "greenRatio": 0.25,
    "parkingRatio": 0.12,
    "accessibilityScore": 0.84,
    "roadEfficiency": 0.78,
    "constraintScore": 1.0
  },
  "buildings": []
}
```

F. APIs consumed

No HTTP APIs.

Python:

```
generate_candidates(...)
```

G. APIs provided

```
validate_layout(
  layout,
  requirements,
  constraints
) -> ValidationResult
```

and:

```
optimize_layouts(
  candidates,
  requirements,
  constraints,
  top_k=5
) -> list[RankedLayout]
```

H. Database

None directly.

I. Dependencies

Person 2

Requires candidate layouts.

Person 1

Requires constraint definitions/schema.

Person 4

Ultimately receives ranked layouts.

J. Mock

```
mock_candidates = [
  {
    "candidateId": "mock-001",
    "siteWidth": 300,
    "siteHeight": 300,
    "buildings": [
      {
        "buildingId": 1,
        "x": 50,
        "y": 50,
        "width": 60,
        "depth": 40,
        "rotation": 0
      }
    ]
  }
]
```

K. Deliverables

- hard constraint engine
- objective functions
- NSGA-II
- Pareto ranking
- top-5 selection
- metrics
- unit tests

- optimizer documentation

L. Definition of Done

Given candidates, the engine returns:

```
Candidate
↓
validated
↓
objectives calculated
↓
optimized
↓
Top 5
```

and no returned plan violates a hard constraint.

M. Tests

- overlap
- boundary
- minimum distance
- zoning
- green
- parking
- roads
- accessibility
- fitness
- Pareto ranking
- optimizer smoke test

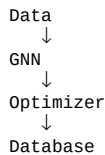
PERSON 4 — BACKEND, DATABASE & SYSTEM ORCHESTRATION

A. Exact Responsibility

Person 4 owns the central application backend.

They connect:

```
Frontend
↓
FastAPI
↓
Database
↓
```



B. Features

FeaturePriority	
-----	--
FastAPI	P0
PostgreSQL	P0
SQLAlchemy	P0
JWT authentication	P0
Project CRUD	P0
Requirements CRUD	P0
Building CRUD	P0
Constraint CRUD	P0
Layout generation	P0
Layout retrieval	P0
Plan selection	P0
Blueprint endpoint	P0
Error handling	P0

C. Files

```
backend/
  main.py
  config.py
  api/
    auth.py
    projects.py
    requirements.py
    buildings.py
    constraints.py
    layouts.py
  models/
    user.py
    project.py
    requirement.py
    building.py
    constraint.py
    layout_run.py
    layout.py
    selected_plan.py
  schemas/
  repositories/
  services/
  tests/
```

D. Inputs

From Person 5:

```
HTTP JSON
```

From Person 1:

```
campus data
```

From Person 2:

```
CandidateLayout[]
```

From Person 3:

```
RankedLayout[]
```

E. Outputs

To frontend:

```
{
  "layouts": []
}
```

To Blender:

```
{
  "projectId": 1,
  "layoutId": 101,
  "site": {},
  "buildings": [],
  "roads": [],
  "greenAreas": [],
  "parkingAreas": [],
  "entrances": []
}
```

F. APIs consumed

Internal Python functions:

```
build_graph()
generate_candidates()
optimize_layouts()
```

No external HTTP APIs.

G. APIs provided

Person 4 owns **all REST APIs** defined in Section 3.

H. Database

Person 4 owns:

```
users
projects
campus_requirements
buildings
constraints
layout_runs
layouts
selected_plans
```

I. Dependencies

PersonRequires	
-----	-----
P1	requirement/building/constraint data
P2	graph builder
P3	optimizer
P5	API availability
P6	blueprint endpoint

J. Mock

Before ML is ready:

```
def mock_generate():
    return [
        {
            "candidateId": "mock-001",
            "rank": 1,
            "feasible": True,
            "metrics": {
                "landUtilization": 0.75,
                "greenRatio": 0.25
            }
        }
    ]
```

K. Deliverables

- database
- migrations

- models
- authentication
- all REST endpoints
- validation
- ML orchestration
- optimizer orchestration
- OpenAPI docs
- tests

L. Definition of Done

A user can:

```

Register
↓
Login
↓
Create project
↓
Enter requirements
↓
Generate layouts
↓
Retrieve layouts
↓
Select layout
↓
Retrieve blueprint

```

using only documented APIs.

M. Tests

- auth
- project CRUD
- requirements
- buildings
- constraints
- layout generation
- layout retrieval
- selection
- blueprint
- authorization
- invalid requests

PERSON 5 — FRONTEND & 2D CAMPUS PLANNING INTERFACE

A. Exact Responsibility

Person 5 owns the actual user interface through which the project manager interacts with the planning system.

B. Features

Feature	Priority
-----	--
Login	P0
Project creation	P0
Site setup	P0
Building setup	P0
Constraint setup	P0
Generate plans	P0
2D visualization	P0
Plan comparison	P0
Metrics	P0
Select plan	P0
Blueprint export	P0

C. Files

```
frontend/  
  ■■■ src/  
    ■■■ pages/  
    ■■■ components/  
    ■■■ api/  
    ■■■ types/  
    ■■■ hooks/  
    ■■■ utils/
```

Person 5 owns all frontend files.

D. Inputs

User input:

```
{  
  "siteWidth": 300,  
  "siteHeight": 300,  
  "minGreenPercent": 25,  
  "minParkingPercent": 10,  
  "minRoadWidth": 8,  
  "minBuildingGap": 10  
}
```

E. Outputs

HTTP JSON requests to Person 4.

2D visualization consumes:

```
{
  "x": 75,
  "y": 120,
  "width": 60,
  "depth": 40,
  "rotation": 0
}
```

F. APIs consumed

All backend APIs required for:

```
authentication
projects
requirements
buildings
constraints
layouts
selection
blueprint
```

G. APIs provided

None.

H. Database

None.

I. Dependencies

Only Person 4's API contracts.

This is intentionally low dependency.

J. Mock

Create:

frontend/src/mocks/layouts.json

with:

```
{
  "layouts": [
    {
      "id": 1,
      "rank": 1,
      "feasible": true,
      "site": {
        "width": 300,
        "height": 300
      },
      "buildings": [
        {
          "id": 1,
          "name": "Academic A",
          "x": 50,
          "y": 100,
          "width": 60,
          "depth": 40,
          "rotation": 0
        }
      ],
      "metrics": {
        "landUtilization": 0.72,
        "greenRatio": 0.25,
        "accessibilityScore": 0.83
      }
    }
  ]
}
```

K. Deliverables

- login
- project setup
- campus form
- building form
- constraints form
- generation workflow
- 2D renderer
- comparison screen
- metrics
- selection
- export

L. Definition of Done

A project manager can complete the entire workflow without touching JSON.

M. Tests

- form validation
- rendering
- rotation
- plan switching
- metrics
- API errors
- selection
- empty states

PERSON 6 — BLENDER AUTOMATION & 3D ENGINEER

A. Exact Responsibility

Person 6 owns the complete 2D blueprint → procedural 3D campus pipeline.

B. Features

Feature	Priority
-----	--
JSON importer	P0
Ground	P0
Buildings	P0
Roads	P0
Parking	P0
Green areas	P0
Entrance	P1
Windows	P1
Trees	P1
Materials	P1
Lighting	P1
Camera	P1
Render	P1

C. Files

```
blender/
■■■ generator.py
■■■ buildings.py
■■■ roads.py
■■■ parking.py
■■■ landscape.py
■■■ materials.py
■■■ lighting.py
■■■ camera.py
■■■ renderer.py
■■■ README.md
```

Person 6 owns all.

D. Inputs

From Person 4:

```
{
  "projectId": 1,
  "layoutId": 101,
  "site": {
    "width": 300,
    "height": 300
  },
  "buildings": [],
  "roads": [],
  "greenAreas": [],
  "parkingAreas": [],
  "entrances": []
}
```

E. Outputs

```
campus.blend
campus_render.png
```

F. APIs consumed

GET /api/layouts/{layoutId}/blueprint

Person 4 → Person 6.

G. APIs provided

None.

H. Database

None.

I. Dependencies

Only the blueprint contract.

J. Mock

Use the existing sample campus JSON and the already-created procedural Blender generator as the initial development target.

The critical requirement is that hard-coded sample coordinates eventually become arbitrary JSON-driven coordinates.

K. Deliverables

- JSON importer
- procedural buildings
- road system
- parking
- green zones
- trees
- entrance
- materials
- lighting
- camera
- automated render
- .blend
- documentation

L. Definition of Done

Running the generator with a different valid blueprint creates the correct campus without manually moving objects.

M. Tests

- JSON parsing
- missing fields
- coordinate mapping

- rotation
- building generation
- road generation
- parking
- vegetation
- rendering
- complete campus generation

3. MASTER API SPECIFICATION

This is the **single source of truth**.

No team member should invent a different endpoint.

API ownership table

Owner	Consumer	Method	Endpoint	Purpose					
-----	----	-----	-----	-----					
P4	P5	POST	/api/auth/register	Register					
P4	P5	POST	/api/auth/login	Login					
P4	P5	POST	/api/projects	Create project					
P4	P5	GET	/api/projects/{projectId}	Get project					
P4	P5	PUT	/api/projects/{projectId}	Update project					
P4	P5	DELETE	/api/projects/{projectId}	Delete project					
P4	P5/P1	POST	/api/projects/{projectId}/requirements	Save requirements					
P4	P5/P1	GET	/api/projects/{projectId}/requirements	Get requirements					
P4	P5/P1	POST	/api/projects/{projectId}/buildings	Add building					
P4	P5/P1	GET	/api/projects/{projectId}/buildings	Get buildings					
P4	P5/P1	POST	/api/projects/{projectId}/constraints	Add constraint					
P4	P5/P1	GET	/api/projects/{projectId}/constraints	Get constraints					
P4	P5	POST	/api/projects/{projectId}/layout-runs	Generate plans					
P4	P5	GET	/api/projects/{projectId}/layouts	List plans					
P4	P5	GET	/api/layouts/{layoutId}	Get plan					
P4	P5	POST	/api/layouts/{layoutId}/select	Select plan					
P4	P6	GET	/api/layouts/{layoutId}/blueprint	Get Blender blueprint					

Common headers

Protected APIs:

```
Authorization: Bearer <JWT>
Content-Type: application/json
```

3.1 Register

POST /api/auth/register

Request:

```
{
  "name": "Student",
  "email": "student@example.com",
  "password": "password123"
}
```

201:

```
{
  "id": 1,
  "name": "Student",
  "email": "student@example.com",
  "role": "PROJECT_MANAGER"
}
```

Errors:

```
400 VALIDATION_ERROR
409 EMAIL_ALREADY_EXISTS
```

3.2 Login

POST /api/auth/login

Request:

```
{
  "email": "student@example.com",
  "password": "password123"
}
```

200:

```
{
  "accessToken": "JWT",
  "tokenType": "Bearer",
  "user": {
    "id": 1,
    "name": "Student",
    "role": "PROJECT_MANAGER"
  }
}
```

Errors:

400 VALIDATION_ERROR
401 INVALID_CREDENTIALS

3.3 Create project

POST /api/projects

Request:

```
{  
  "name": "VIT Campus Expansion",  
  "description": "Campus planning case study"  
}
```

201:

```
{  
  "id": 1,  
  "name": "VIT Campus Expansion",  
  "description": "Campus planning case study",  
  "status": "DRAFT"  
}
```

3.4 Get project

GET /api/projects/{projectId}

200:

```
{  
  "id": 1,  
  "name": "VIT Campus Expansion",  
  "description": "Campus planning case study",  
  "status": "DRAFT"  
}
```

Errors:

401
403
404

3.5 Update project

PUT /api/projects/{projectId}

Request:

```
{
  "name": "Updated Campus Expansion",
  "description": "Updated description"
}
```

200:

```
{
  "id": 1,
  "name": "Updated Campus Expansion",
  "description": "Updated description",
  "status": "DRAFT"
}
```

3.6 Delete project

DELETE /api/projects/{projectId}

204

No response body.

3.7 Save requirements

POST /api/projects/{projectId}/requirements

Request:

```
{
  "siteWidth": 300,
  "siteHeight": 300,
  "minGreenPercent": 25,
  "minParkingPercent": 10,
  "minRoadWidth": 8,
  "minBuildingGap": 10,
  "entrances": [
    {
      "x": 150,
      "y": 0,
      "width": 10
    }
  ]
}
```

201:

```
{
  "id": 1,
  "projectId": 1,
  "siteWidth": 300,
  "siteHeight": 300,
  "minGreenPercent": 25,
  "minParkingPercent": 10,
  "minRoadWidth": 8,
  "minBuildingGap": 10,
  "entrances": [
    {
      "x": 150,
      "y": 0,
      "width": 10
    }
  ]
}
```

```
    }  
  ]  
}
```

3.8 Get requirements

GET /api/projects/{projectId}/requirements

200:

```
{  
  "projectId": 1,  
  "siteWidth": 300,  
  "siteHeight": 300,  
  "minGreenPercent": 25,  
  "minParkingPercent": 10,  
  "minRoadWidth": 8,  
  "minBuildingGap": 10,  
  "entrances": []  
}
```

3.9 Add building

POST /api/projects/{projectId}/buildings

Request:

```
{  
  "name": "Academic Block A",  
  "type": "academic",  
  "zone": "academic",  
  "width": 60,  
  "depth": 40,  
  "height": 18,  
  "floorCount": 4,  
  "requiredCount": 1  
}
```

201:

```
{  
  "id": 1,  
  "projectId": 1,  
  "name": "Academic Block A",  
  "type": "academic",  
  "zone": "academic",  
  "width": 60,  
  "depth": 40,  
  "height": 18,  
  "floorCount": 4,  
  "requiredCount": 1  
}
```

3.10 Get buildings

GET /api/projects/{projectId}/buildings

200:

```
{
  "buildings": [
    {
      "id": 1,
      "name": "Academic Block A",
      "type": "academic",
      "zone": "academic",
      "width": 60,
      "depth": 40,
      "height": 18,
      "floorCount": 4,
      "requiredCount": 1
    }
  ]
}
```

3.11 Add constraint

POST /api/projects/{projectId}/constraints

Request:

```
{
  "type": "MIN_DISTANCE",
  "sourceId": 1,
  "targetId": 5,
  "value": 100,
  "operator": ">=",
  "priority": "hard"
}
```

201:

```
{
  "id": 1,
  "type": "MIN_DISTANCE",
  "sourceId": 1,
  "targetId": 5,
  "value": 100,
  "operator": ">=",
  "priority": "hard"
}
```

3.12 Get constraints

GET /api/projects/{projectId}/constraints

200:

```
{
  "constraints": [
    {
      "id": 1,
      "type": "MIN_DISTANCE",
      "sourceId": 1,
      "targetId": 5,
      "value": 100,
      "operator": ">=",
      "priority": "hard"
    }
  ]
}
```

```
    }  
  ]  
}
```

3.13 Generate layouts

POST /api/projects/{projectId}/layout-runs

Request:

```
{  
  "candidateCount": 100,  
  "topK": 5,  
  "algorithm": "GNN_NSQA2"  
}
```

200:

```
{  
  "runId": 12,  
  "status": "COMPLETED",  
  "layoutCount": 5  
}
```

Errors:

```
400 INVALID_REQUIREMENTS  
404 PROJECT_NOT_FOUND  
422 GENERATION_FAILED  
500 INTERNAL_SERVER_ERROR
```

3.14 List layouts

GET /api/projects/{projectId}/layouts

200:

```
{  
  "layouts": [  
    {  
      "id": 101,  
      "rank": 1,  
      "feasible": true,  
      "metrics": {  
        "landUtilization": 0.72,  
        "greenRatio": 0.25,  
        "parkingRatio": 0.12,  
        "accessibilityScore": 0.84,  
        "roadEfficiency": 0.78,  
        "constraintScore": 1.0  
      }  
    }  
  ]  
}
```

3.15 Get layout

```
GET /api/layouts/{layoutId}
```

200:

```
{
  "id": 101,
  "rank": 1,
  "feasible": true,
  "site": {
    "width": 300,
    "height": 300
  },
  "buildings": [],
  "roads": [],
  "greenAreas": [],
  "parkingAreas": [],
  "entrances": [],
  "metrics": {}
}
```

3.16 Select layout

```
POST /api/layouts/{layoutId}/select
```

Request:

```
{}
```

200:

```
{
  "projectId": 1,
  "layoutId": 101,
  "status": "SELECTED"
}
```

3.17 Blender blueprint

```
GET /api/layouts/{layoutId}/blueprint
```

200:

```
{
  "projectId": 1,
  "layoutId": 101,
  "site": {
    "width": 300,
    "height": 300
  },
  "buildings": [
    {
      "id": 1,
      "name": "Academic Block A",
      "type": "academic",
      "zone": "academic",
      "x": 75,
      "y": 120,
      "width": 60,
      "depth": 40,

```

```

        "height": 18,
        "rotation": 0,
        "floorCount": 4
    }
],
"roads": [],
"greenAreas": [],
"parkingAreas": [],
"entrances": []
}

```

4. MASTER DATA CONTRACTS

These should be placed in a shared:

contracts/

directory.

Only the contract owner should modify them.

Contract 1 — Building

```

interface Building {
    id: number;
    name: string;
    type: string;
    zone: string;
    width: number;
    depth: number;
    height: number;
    floorCount: number;
    requiredCount: number;
}

```

Contract 2 — Campus Requirements

```

interface CampusRequirements {
    projectId: number;
    siteWidth: number;
    siteHeight: number;
    minGreenPercent: number;
    minParkingPercent: number;
    minRoadWidth: number;
    minBuildingGap: number;
    entrances: Entrance[];
}

```

Contract 3 — Entrance

```

interface Entrance {
    x: number;
    y: number;
    width: number;
}

```

Contract 4 — Spatial relationship

```
interface SpatialRelationship {
  sourceId: number;
  targetId: number;
  relation:
    | "NEAR"
    | "FAR"
    | "SAME_ZONE"
    | "ACCESSIBLE_FROM"
    | "ROAD_ACCESS";
  weight: number;
}
```

Contract 5 — Campus graph

Producer: P1

Consumer: P2

```
interface CampusGraph {
  nodeFeatures: number[][];
  edgeIndex: number[][];
  edgeFeatures: number[][];
}
```

Contract 6 — Candidate layout

Producer: P2

Consumer: P3

```
interface CandidateBuilding {
  buildingId: number;
  x: number;
  y: number;
  rotation: number;
}

interface CandidateLayout {
  candidateId: string;
  siteWidth: number;
  siteHeight: number;
  buildings: CandidateBuilding[];
}
```

Contract 7 — Validation result

Producer: P3

```
interface ValidationResult {
  feasible: boolean;
}
```

```

    violations: {
      type: string;
      buildingIds: number[];
      message: string;
      severity: "hard" | "soft";
    }[];
    constraintScore: number;
  }

```

Contract 8 — Ranked layout

Producer: P3

Consumer: P4

```

interface RankedLayout {
  candidateId: string;
  rank: number;
  feasible: boolean;

  buildings: CandidateBuilding[];

  metrics: {
    landUtilization: number;
    greenRatio: number;
    parkingRatio: number;
    accessibilityScore: number;
    roadEfficiency: number;
    constraintScore: number;
  };
}

```

Contract 9 — Blueprint

Producer: P4

Consumer: P6

```

interface Blueprint {
  projectId: number;
  layoutId: number;

  site: {
    width: number;
    height: number;
  };

  buildings: BlueprintBuilding[];
  roads: Road[];
  greenAreas: Area[];
  parkingAreas: Area[];
  entrances: Entrance[];
}

```

5. DATABASE CONTRACT

Relationships

```

User
  ■
  ■■■< Projects
    ■
    ■■■< Buildings
    ■
    ■■■< Constraints
    ■
    ■■■< LayoutRuns
    ■
    ■ ■■■< Layouts
    ■
    ■■■ SelectedPlan

```

users

```

id
name
email
password_hash
role
created_at

```

projects

```

id
user_id
name
description
status
created_at
updated_at

```

Relationship:

```

users.id → projects.user_id

```

campus_requirements

```

id
project_id
site_width
site_height
total_area
min_green_percent
min_parking_percent
min_road_width
min_building_gap
entrance_data JSONB
road_data JSONB

```

buildings

```

id

```

```
project_id
name
type
zone
width
depth
height
floor_count
required_count
metadata JSONB
```

constraints

```
id
project_id
constraint_type
source_id
target_id
value
operator
priority
metadata JSONB
```

layout_runs

```
id
project_id
algorithm
population_size
generation_count
status
created_at
completed_at
```

layouts

```
id
run_id
rank
feasible
layout_json JSONB
metrics_json JSONB
created_at
```

selected_plans

```
id
project_id
layout_id
selected_by
selected_at
```

6. DEPENDENCY MATRIX

Module-level

Module	Depends on	What it needs	Mockable?
-----	-----	-----	-
P1 Data	P4	Requirements/buildings/constraints	■
P2 GNN	P1	CampusGraph	■
P3 Optimization	P2/P1	Candidates + constraints	■
P4 Backend	P1/P2/P3	Python interfaces	■
P5 Frontend	P4	REST contracts	■
P6 Blender	P4	Blueprint JSON	■

6×6 dependency matrix

→ means row depends on column.

P1	P2	P3	P4	P5	P6
-----	-	-	-	-	-
P1	—	→	→	→	—
P2	→	—	→	—	—
P3	→	→	—	—	—
P4	→	→	→	—	→
P5	—	—	—	→	—
P6	—	—	—	→	—

Most important observation

There are **no circular dependencies**.

```

P1
↓
P2
↓
P3
↓
P4
↓
P5

P4 → P6
  
```

The frontend and Blender can be developed almost completely independently.

7. PARALLEL DEVELOPMENT STRATEGY

The team should **not wait for the AI model to be finished** before starting.

What everyone can start TODAY

Person 1

Day 1

- Create schemas
- Create synthetic campus generator
- Define graph representation
- Create 50 sample campuses
- Validate dataset

After contracts

- Generate 1,000+ samples
- Implement graph builder
- Export training data

Integration

- Connect graph builder to P2

Person 2

Day 1

- Create GNN architecture
- Create mock graph
- Create decoder
- Build forward pass

After contracts

- Connect real CampusGraph
- Train model
- Save checkpoint

Integration

- Connect generate_candidates() to P3

Person 3

Day 1

- Implement rectangle geometry
- Boundary check
- Overlap check
- Distance check
- Objective functions

After contracts

- Implement NSGA-II
- Connect P2 candidates

Integration

- Validate real GNN outputs

Person 4

Day 1

- Create FastAPI
- Create PostgreSQL
- Create SQLAlchemy models
- Create migrations
- Implement auth

After contracts
- Implement all REST APIs
- Add mock ML pipeline

Integration
- Replace mock with P1/P2/P3

Person 5

Day 1
- React project
- Build login screen
- Build project form
- Build 2D canvas
- Use mock JSON

After contracts
- Connect Axios APIs
- Display actual layouts

Integration
- Test complete user workflow

Person 6

Day 1
- Set up Blender
- Refactor existing generator
- Implement JSON parser
- Generate buildings
- Generate roads

After contracts
- Match Blueprint schema

Integration
- Fetch selected blueprint
- Generate final 3D campus

Interfaces to freeze FIRST

These six should be frozen before serious parallel development:

1. CampusRequirements

P4 → P1

2. Building

P4 → P1

3. CampusGraph

P1 → P2

4. CandidateLayout

P2 → P3

5. RankedLayout

P3 → P4

6. Blueprint

P4 → P6

These are the **six critical interfaces**.

8. GIT STRATEGY

Repository

Use one monorepo:

```
ai-campus-planner/  
■  
■■■ frontend/  
■■■ backend/  
■■■ ml/  
■■■ optimization/  
■■■ blender/  
■■■ contracts/  
■■■ docs/  
■■■ tests/  
■■■ docker-compose.yml  
■■■ README.md  
■■■ .gitignore
```

Branches

```
main  
integration  
  
feature/p1-data  
feature/p2-gnn  
feature/p3-optimization  
feature/p4-backend  
feature/p5-frontend  
feature/p6-blender
```

Ownership

DirectoryOwner	
-----	-----

ml/data	P1
ml/dataset	P1
ml/models	P2
ml/training	P2
ml/inference	P2
optimization	P3
backend	P4
frontend	P5
blender	P6
contracts	Team agreement; designated maintainer P4

Commit convention

Use:

```
feat:
fix:
test:
refactor:
docs:
chore:
```

Examples:

```
feat: add synthetic campus generator
feat: implement GNN layout decoder
feat: add minimum-distance constraint
feat: add project creation API
feat: add 2D layout viewer
feat: add Blender JSON importer
```

PR rules

Every PR must:

1. modify only the owner's area where possible,
2. pass tests,
3. contain a useful commit message,
4. not change contracts without team approval.

Merge strategy

```
Person branch
↓
Pull Request
↓
integration
↓
```

```
automated tests
  ↓
main
```

Do not develop directly on main.

Conflict avoidance

Particularly important:

Nobody should edit:

```
contracts/layout.ts
```

casually.

If the contract changes:

```
PR
  ↓
team approval
  ↓
version update
  ↓
all affected modules updated
```

9. INTEGRATION ORDER

STEP 1 — Contracts

Owner: P4 coordinates

Participants: all

Freeze:

```
CampusRequirements
Building
Constraint
CampusGraph
CandidateLayout
RankedLayout
Blueprint
```

Verification:

```
All example JSON files validate.
```

STEP 2 — Database + Backend

Owner: P4

Connect:

```
React-independent backend
  ↓
PostgreSQL
```

Test:

```
POST project
GET project
POST buildings
GET buildings
POST constraints
GET constraints
```

STEP 3 — Data → GNN

Owners: P1 + P2

Connect:

```
P1 graph_builder
  ↓
CampusGraph
  ↓
P2 GNN
```

Test:

```
graph = build_graph(project)
candidates = generate_candidates(graph, 100)
```

Expected:

```
100 CandidateLayout objects
```

STEP 4 — GNN → Optimization

Owners: P2 + P3

Connect:

```
CandidateLayout[]
  ↓
Constraint Engine
  ↓
NSGA-II
  ↓
RankedLayout[]
```

Expected:

Top 5 feasible plans

STEP 5 — ML → Backend

Owners: P3 + P4

Connect:

```
POST /api/projects/{id}/layout-runs
  ↓
P4 orchestration
  ↓
P1
  ↓
P2
  ↓
P3
  ↓
PostgreSQL
```

Expected:

```
{
  "runId": 12,
  "status": "COMPLETED",
  "layoutCount": 5
}
```

STEP 6 — Backend → Frontend

Owners: P4 + P5

Connect:

```
React
  ↓
POST layout-runs
  ↓
FastAPI
  ↓
GET layouts
  ↓
React 2D viewer
```

Expected:

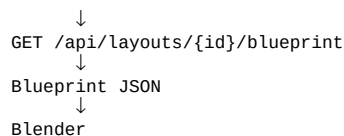
Five actual campus plans appear on screen.

STEP 7 — Backend → Blender

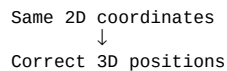
Owners: P4 + P6

Connect:

Selected Layout

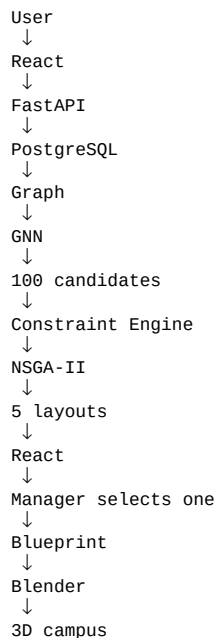


Expected:



STEP 8 — Full E2E

Entire system:



10. BLOCKER ANALYSIS

ProblemWhy it blocksSolutionFallback			
-----	-----	-----	-----
GNN not trained	P3 has no candidates	P2 supplies mock candidates	Random layout generator
Dataset incomplete	P2 cannot train	P1 uses synthetic generator	Small 50-sample dataset
Backend unfinished	P5 cannot connect	Freeze REST contracts	Mock JSON
Blender unfinished	Final demo blocked	P6 uses sample blueprint	Existing Blender prototype
NSGA-II unfinished	No ranking	P3 uses weighted fitness	Sort by constraint score
Database unfinished	Backend blocked	SQLite temporary	JSON persistence
API changes	Multiple modules break	Version contracts	Contract review

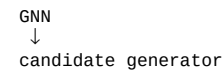
GNN produces invalid coordinates	Plans fail	P3 validates/repairs	Randomized candidate generation
GPU unavailable	Training slow	CPU/smaller model	Smaller dataset/model
Blender version differences	Scripts fail	Pin Blender version	Test on one agreed version

Highest-risk blockers

■ 1. GNN training

Do **not** make the whole project dependent on the GNN being perfect.

Our fallback:



can initially be replaced by:

random/heuristic candidate generator

The rest of the architecture remains unchanged.

This is extremely important.

■ 2. Dataset

Do not wait for "real campus data."

Generate synthetic data immediately.

■ 3. Blender

The Blender component must be developed using static JSON first.

Do not wait for AI.

■ 4. Frontend

Build using mocked layouts.

Do not wait for FastAPI.

11. RECOMMENDED MVP

Do **not** attempt the entire hypothetical VIT-AP campus immediately.

Start with:

```
300 × 300 m plot

2 Academic Buildings
3 Hostels
1 Library
1 Administration
1 Sports Complex
1 Parking Area
1 Green Area
1 Main Entrance
Road network
```

Pipeline:

```
Input
↓
Generate 100 candidates
↓
Constraint validation
↓
NSGA-II
↓
Top 5
↓
2D visualization
↓
Select one
↓
JSON
↓
Blender
↓
3D
```

Once this works, increase complexity.

12. WHAT "AI" ACTUALLY DOES

This distinction is important when presenting to your professor.

Person 1

Transforms:

```
Campus requirements
→ graph/data
```

Person 2

AI learns:

```
Which buildings should have which spatial relationships
```

and predicts:

x, y, rotation

Person 3

Determines:

Is the generated plan feasible?
Which plans are better?

Person 4

Turns everything into:

A usable software system

Person 5

Lets the manager:

Interact with and compare plans

Person 6

Turns:

2D AI-generated blueprint
→ 3D campus

13. FINAL INDIVIDUAL HANDOFF DOCUMENTS

These can be copied directly to each team member.

PERSON 1 — DATASET & CAMPUS REPRESENTATION

1. Responsibility

Build the complete data representation and synthetic dataset pipeline for campus planning.

2. Features

- Campus schema
- Building schema
- Constraints

- Spatial relationships
- Synthetic data generator
- Graph construction
- Dataset validation

3. Files

```
ml/data/*
ml/dataset/*
ml/datasets/*
```

4. Inputs

```
CampusRequirements
Building[]
Constraint[]
```

5. Outputs

```
CampusGraph
Dataset records
```

6. APIs consumed

```
GET /api/projects/{projectId}/requirements
GET /api/projects/{projectId}/buildings
GET /api/projects/{projectId}/constraints
```

7. APIs provided

None.

Python:

```
build_graph(campus_data) -> CampusGraph
```

8. Database

Read:

```
projects
campus_requirements
buildings
constraints
```

Do not write DB.

9. Dependencies

P4 supplies structured project data.

P2 consumes your CampusGraph.

10. Mock

Use static campus JSON.

11. Deliverables

- schemas
- graph builder
- synthetic generator
- 1,000+ samples
- validator
- documentation

12. Tests

Schema, graph, dataset, malformed-input tests.

13. Definition of Done

P2 can directly train/infer using your graph without modifying it.

14. First things to implement

1. campus_schema.py
2. building_schema.py
3. constraint_schema.py
4. relationships.py
5. generator.py
6. graph_builder.py

PERSON 2 — GNN & LAYOUT GENERATOR

1. Responsibility

Build the DL model that converts campus graphs into candidate layouts.

2. Features

- GNN
- graph embeddings
- decoder
- coordinate prediction
- rotation
- training
- inference
- candidate generation

3. Files

```
ml/models/*
ml/training/*
ml/inference/*
```

4. Inputs

CampusGraph

5. Outputs

CandidateLayout[]

6. APIs consumed

No HTTP APIs.

7. APIs provided

```
generate_candidates(  
    campus_graph,  
    num_candidates  
) -> List[CandidateLayout]
```

8. Database

None.

9. Dependencies

P1:

CampusGraph

P3 consumes your candidates.

10. Mock

Static graph → static candidate layout.

11. Deliverables

- GNN
- decoder
- training
- checkpoint
- inference
- candidate generator

12. Tests

Forward pass, training, checkpoint, inference, schema.

13. Definition of Done

Model can accept unseen graph data and generate candidate layouts.

14. First things to implement

1. gnn_encoder.py
2. layout_decoder.py
3. campus_model.py
4. loss.py
5. train.py
6. generate.py

PERSON 3 — CONSTRAINT & NSGA-II ENGINE

1. Responsibility

Validate generated layouts and select the best feasible plans using multi-objective optimization.

2. Features

- Boundary
- Overlap
- Distance
- Zoning
- Roads
- Parking
- Green space
- Accessibility
- Objectives
- NSGA-II
- Pareto ranking

3. Files

optimization/*

4. Inputs

CandidateLayout[]
CampusRequirements
Constraint[]

5. Outputs

RankedLayout[]

6. APIs consumed

No HTTP APIs.

7. APIs provided

```
validate_layout(...)  
optimize_layouts(...)
```

8. Database

None.

9. Dependencies

P2 → candidate layouts.

P1 → constraint representation.

10. Mock

Use manually created candidate layouts.

11. Deliverables

- validator
- all hard constraints
- objectives
- NSGA-II
- top-5 ranking
- metrics

12. Tests

Geometry, constraints, objectives, optimizer, ranking.

13. Definition of Done

Input:

```
100 candidates
```

Output:

```
5 feasible/ranked plans
```

14. First things to implement

1. overlap.py
2. boundary.py

```
3. distance.py
4. zoning.py
5. objectives.py
6. validator.py
7. nsga2.py
8. optimizer.py
```

PERSON 4 — BACKEND, DATABASE & ORCHESTRATION

1. Responsibility

Build the central FastAPI application, PostgreSQL database and connect the AI pipeline.

2. Features

- Authentication
- Projects
- Requirements
- Buildings
- Constraints
- Layout runs
- Layout retrieval
- Plan selection
- Blueprint export

3. Files

```
backend/*
```

4. Inputs

Frontend JSON.

ML:

```
CampusGraph
CandidateLayout[]
RankedLayout[]
```

5. Outputs

REST JSON APIs.

6. APIs consumed

Internal:

```
build_graph()
generate_candidates()
```

optimize_layouts()

7. APIs provided

All:

```
POST /api/auth/register
POST /api/auth/login

POST /api/projects
GET /api/projects/{id}
PUT /api/projects/{id}
DELETE /api/projects/{id}

POST /api/projects/{id}/requirements
GET /api/projects/{id}/requirements

POST /api/projects/{id}/buildings
GET /api/projects/{id}/buildings

POST /api/projects/{id}/constraints
GET /api/projects/{id}/constraints

POST /api/projects/{id}/layout-runs
GET /api/projects/{id}/layouts

GET /api/layouts/{id}
POST /api/layouts/{id}/select
GET /api/layouts/{id}/blueprint
```

8. Database

Own all tables.

9. Dependencies

P1/P2/P3 Python interfaces.

P5 consumes REST APIs.

P6 consumes blueprint API.

10. Mock

Mock ML pipeline returning five layouts.

11. Deliverables

- PostgreSQL
- FastAPI
- SQLAlchemy
- authentication
- APIs
- ML orchestration
- tests

12. Tests

All endpoint and authorization tests.

13. Definition of Done

The complete application workflow works through REST APIs.

14. First things to implement

1. PostgreSQL
2. SQLAlchemy models
3. migrations
4. authentication
5. project APIs
6. requirements APIs
7. building APIs
8. constraint APIs

Then layout APIs.

PERSON 5 — FRONTEND & 2D PLANNER

1. Responsibility

Build the project-manager interface for configuring campuses and comparing generated layouts.

2. Features

- Login
- Project creation
- Site input
- Building input
- Constraints
- Generate
- 2D viewer
- Plan comparison
- Metrics
- Selection

3. Files

frontend/*

4. Inputs

User-entered campus data.

5. Outputs

REST requests and visual layouts.

6. APIs consumed

All P4 frontend APIs.

7. APIs provided

None.

8. Database

None.

9. Dependencies

P4 REST contracts only.

10. Mock

```
frontend/src/mocks/layouts.json
```

11. Deliverables

Complete planning UI.

12. Tests

Forms, rendering, API errors, plan selection.

13. Definition of Done

Manager can create → generate → compare → select a campus plan.

14. First things to implement

1. React/Vite
2. routing
3. login
4. project page
5. requirements form
6. building form
7. 2D viewer

PERSON 6 — BLENDER 3D AUTOMATION

1. Responsibility

Convert the selected blueprint JSON into an automated 3D campus.

2. Features

- JSON import
- buildings
- roads

- parking
- green spaces
- entrance
- trees
- materials
- lighting
- camera
- rendering

3. Files

blender/*

4. Inputs

Blueprint JSON

5. Outputs

.blend
.png

6. APIs consumed

GET /api/layouts/{layoutId}/blueprint

7. APIs provided

None.

8. Database

None.

9. Dependencies

Only P4's Blueprint contract.

10. Mock

Use the existing sample campus JSON and current procedural Blender prototype.

11. Deliverables

- importer
- building generator
- roads
- parking

- landscaping
- materials
- lighting
- rendering

12. Tests

JSON, coordinates, rotation, generation and render tests.

13. Definition of Done

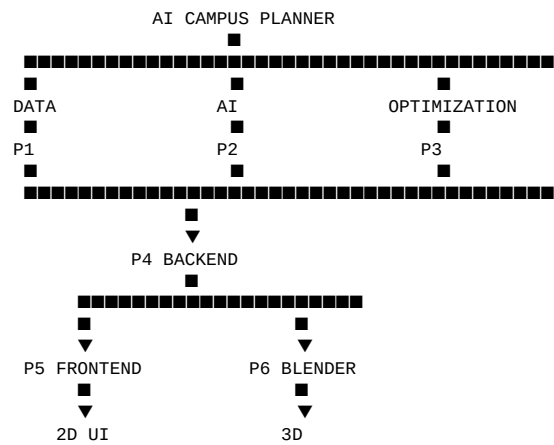
A completely different valid blueprint generates the correct 3D campus automatically.

14. First things to implement

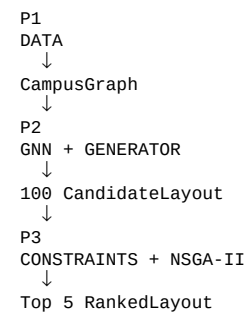
1. JSON importer
2. coordinate conversion
3. building generator
4. road generator
5. parking
6. landscape
7. camera
8. renderer

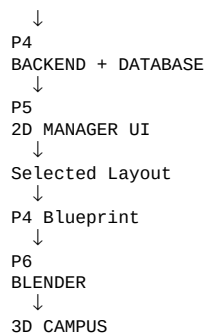
FINAL TEAM STRUCTURE

Your six-person team should therefore look like this:



And the **critical pipeline** is:





Most important rule for the team

Nobody waits for another person to finish before starting.

Start with mocks and freeze the six interfaces:

```
CampusRequirements
Building
CampusGraph
CandidateLayout
RankedLayout
Blueprint
```

If those six contracts remain stable, the six people can work almost completely independently and integration becomes a matter of **connecting modules**, rather than rewriting them.

Recommended difficulty/skill matching from our earlier assessment:

1. **P2 — GNN/Layout Generation:** hardest ML/research
2. **P3 — Constraints/NSGA-II:** hardest algorithms
3. **P4 — Backend/Orchestration:** highest integration responsibility
4. **P6 — Blender:** specialized Python/3D work
5. **P1 — Dataset/Graph:** data + ML foundation
6. **P5 — Frontend:** comparatively straightforward, but essential for the actual product

This gives you a realistic division where **every person owns a complete subsystem**, no two people own the same code, and the project can still demonstrate a functioning MVP even if the GNN isn't perfect.

svg

svg

svg

svg